

16 | tokio编程：使用channel在不同任务间通信？

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们来了解并发编程的另一种范式——使用 channel 在不同的任务间进行通信。

channel 翻译成中文就是通道或管道，用来在 task 之间传递消息。这个概念本身并不难。我们回忆一下上节课的目标：要在多个任务中同时对一个内存数据库进行更新。其实我们也可以用 channel 的思路来解决这个问题。

我们先来分解一下任务。

1. 创建三个子任务，task_a、task_b 和另一个起代理作用的 task_c。
2. 在 task_a 和 task_b 中，不直接操作 db 本身，而是向 task_c 发一个消息。
3. task_c 里面会拿到 db 的所有权，收到从 task_a 和 task_b 来的消息后，对 db 进行操作。

基于这个思路，我们来重写上一节课的示例。

MPSC Channel

我们使用 tokio 中的 MPSC Channel 来实现。MPSC Channel 是多生产者，单消费者通道 (Multi-Producers Single Consumer) 。

MPSC 的基本用法如下：

[复制代码](#)

```
1 let (tx, mut rx) = mpsc::channel(100);
```

使用 MPSC 模块的 `channel()` 函数创建一个通道对，tx 表示发送端，rx 表示接收端，rx 前面要加 `mut` 修饰符，因为 rx 在接收数据的时候使用了可变借用。channel 使用的时候要给一个整数参数，表示这个通道容量多大。tokio 的这个 `mpsc::channel` 是带背压功能的，也就是说，如果发送端发得太快，接收端来不及消耗导致通道堵塞了的话，这个 channel 会让发送端阻塞等待，直到通道里面的数据包被消耗到留出空位为止。

MPSC 的特点就是可以有多个生产者，但只有一个消费者。因此，tx 可以被随意 clone 多份，但是 rx 只能有一个。

前面的例子，我们用 channel 来实现。

[复制代码](#)

```
1 use tokio::sync::mpsc;
2
3 #[tokio::main]
4 async fn main() {
5     let mut db: Vec<u32> = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
6     let (tx, mut rx) = mpsc::channel::<u32>(100); // 创建channel
7
8     let tx1 = tx.clone(); // 拷贝两份arc
9     let tx2 = tx.clone();
10
11     let task_a = tokio::task::spawn(async move {
12         if let Err(_) = tx1.send(50).await { // 发送端标准写法
13             println!("receiver dropped");
14             return;
15         }
16     });
```

```

16     });
17     let task_b = tokio::task::spawn(async move {
18         if let Err(_) = tx2.send(100).await { // 发送端标准写法
19             println!("receiver dropped");
20             return;
21         }
22     });
23
24     let task_c = tokio::task::spawn(async move {
25         while let Some(i) = rx.recv().await { // 接收端标准写法
26             println!("got = {}", i);
27             db[4] = i;
28             println!("{:?}", db);
29         }
30     });
31     _ = task_a.await.unwrap();
32     _ = task_b.await.unwrap();
33     _ = task_c.await.unwrap();
34 }
35 //输出
36 got = 50
37 [1, 2, 3, 4, 50, 6, 7, 8, 9, 10]
38 got = 100
39 [1, 2, 3, 4, 100, 6, 7, 8, 9, 10]
40 ^C

```

代码第 6 行，我们使用 `let (tx, mut rx) = mpsc::channel:::<u32>(100);` 创建一个 channel，注意，这一句使用 `::<u32>` 指定了这个 channel 中要传输的消息类型，也就是传 u32 类型的整数，通道容量为 100 个消息。

第 8 行和第 9 行，clone 了两份 tx。因为 tx 本质上实现为一个 Arc 对象，因此 clone 它也就只增加了引用计数，没有多余的性能消耗。

第 11 行和第 17 行，创建了两个工作者任务，在里面我们用 `if let Err(_) = tx1.send(50).await` 这种写法来向 channel 中发送信息，因为向 MPSC Channel 中灌数据时，是有可能出错的，比如 channel 的另一端 rx 已经关闭了（被释放了），那么这时候再用 tx 发数据就会产生一个错误，所以这里需要用 `if let Err(_) =` 这种形式来处理错误。

第 24 行，创建一个代理任务 task_c，使用这种写法 `while let Some(i) = rx.recv().await` 来接收消息。这里 `rx.recv().await` 获取回来的是一个

`Option<u32>` 类型，因此要用 `while let Some(i)` 这种模式匹配语法来写，`i` 就是收到的消息。然后在 `while` 内部处理具体的业务就行了。当 `rx` 收到一个 `None` 值（`channel` 关闭产生的）的时候，会退出这个循环。

可以看到，当业务正常进行时，这个程序不会自动终止，而是会一直处于工作状态，最后我们得用 `Ctrl-C` 在终端终止它的运行。为什么呢？因为 `while let` 没有退出。`rx.recv().await` 一直在等待下一个 `msg` 的到来，但是前面两个发消息的任务 `task_a`、`task_b` 的工作已经完成，退出了，于是没有角色给 `rx` 发消息了，它就会一直等下去。这里的 `.await` 是一种不消耗资源的等待，`tokio` 保证这种等待不会让一个 CPU 忙空转。

第 31 行 ~ 第 33 行的顺序在这里并不是很重要，你可以试试改变 `task_a`、`task_b`、`task_c` 的 `await` 的顺序，看看输出结果的变化。

花几分钟理解了这个过程后，你会发现这个方案的思维方式和前面使用锁的方式完全不同。这其实是一种常见的设计模式：代理模式。

真正的并发执行

`tokio::task::spawn()` 这个 API 有个特点，就是通过它创建的异步任务，一旦创建好，就会立即扔到 `tokio runtime` 里执行，不需要对其返回的 `JoinHandler` 进行 `await` 才驱动执行，这个特性很重要。

我们使用这个特性分析一下前面的示例：`task_a`、`task_b`、`task_c` 创建好之后，实际就已经开始执行了。`task_c` 已经在等待 `channel` 数据的到来了。第 31 到 33 行 `JoinHandler` 的 `await` 只是在等待任务本身结束而已。我们试着修改一下上面的示例。

[复制代码](#)

```
1 use std::time::Duration;
2 use tokio::sync::mpsc;
3 use tokio::task;
4 use tokio::time;
5
6 #[tokio::main]
7 async fn main() {
8     let mut db: Vec<u32> = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
9     let (tx, mut rx) = mpsc::channel::<u32>(100);
```

```

10
11     let tx1 = tx.clone();
12     let tx2 = tx.clone();
13
14     let task_a = task::spawn(async move {
15         println!("in task_a 1");
16         time::sleep(Duration::from_secs(3)).await; // 等待3s
17         println!("in task_a 2");
18         if let Err(_) = tx1.send(50).await {
19             println!("receiver dropped");
20             return;
21         }
22     });
23     let task_b = task::spawn(async move {
24         println!("in task_b");
25         if let Err(_) = tx2.send(100).await {
26             println!("receiver dropped");
27             return;
28         }
29     });
30
31     let task_c = task::spawn(async move {
32         while let Some(i) = rx.recv().await {
33             println!("got = {}", i);
34             db[4] = i;
35             println!("{:?}", db);
36         }
37     });
38
39     _ = task_c.await.unwrap(); // task_c 放在前面来await
40     _ = task_a.await.unwrap();
41     _ = task_b.await.unwrap();
42 }
43 // 输出
44 in task_a 1
45 in task_b
46 got = 100
47 [1, 2, 3, 4, 100, 6, 7, 8, 9, 10]
48 in task_a 2
49 got = 50
50 [1, 2, 3, 4, 50, 6, 7, 8, 9, 10]
51 ^C

```

在这个示例里，我们在 task_a 中 sleep 了 3 秒（第 16 行）。同时把 task_c 放到最前面去 await 了（第 39 行）。可以看到，task_b 发来的数据先打印，3 秒后，task_a 发来的数据打

印了。

实际对于 main 函数这个 task 来讲，它其实被阻塞在了第 39 行，因为 task_c 一直在 await，并没有结束。task_a 和 task_b 虽然已经结束了，但是并没有执行到第 40 行和第 41 行去。对整个程序的输出来讲，没有执行到第 40 行和第 41 行并不影响最终效果。你仔细体会一下。

所以使用 `task::spawn()` 创建的多个任务之间，本身就是并发执行的关系。你可以对比一下这两个示例。

unbounded channel

tokio::mpsc 模块里还有一个函数 `mpsc::unbounded_channel()`，可以用来创建没有容量上限的通道，也就意味着，它不具有背压功能。这个通道里面能存多少数据，就看机器的内存多大，极端情况下，可能会撑爆你的服务器。而在使用方法上，这两种 channel 区别不大，因此不再举例说明。如果你感兴趣的话可以看一下我给出的 [🔗 链接](#)。

Oneshot Channel

如果现在我们要在前面示例的基础上增加一个需求：我在 task_c 中将 db 更新完成，想给 task_a 和 task_b 返回一个事件通知说，我已经完成了，应该怎么做？

这个问题当然不止一种解法，比如外部增加一个消息队列，将这两个消息抛进消息队列里面，让 task_a 和 task_b 监听这个队列。然而这个方案会增加对外部服务的依赖，可能是一个订阅 - 发布服务；task_a 和 task_b 里需要订阅外部消息队列，并过滤对应的消息进行处理。

tokio 其实内置了另外一个好用的东西 Oneshot channel，它可以配合 MPSC Channel 完成我们的任务。Oneshot 定义了这样一个模型，这个通道只能用一次，也就是说只能发送一条数据，发送完之后就关闭了，对应的 tx 和 rx 就无法再次使用了。这个很适合等待计算结果返回的场景。我们试着用这个新设施来实现一下我们的需求。

 复制代码

```
1 use std::time::Duration;  
2 use tokio::sync::{mpsc, oneshot};
```



```
3 use tokio::task;
4 use tokio::time;
5
6 #[tokio::main]
7 async fn main() {
8     let mut db: Vec<u32> = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
9     let (tx, mut rx) = mpsc::channel::<(u32, oneshot::Sender<bool>)>(100);
10
11     let tx1 = tx.clone();
12     let tx2 = tx.clone();
13
14     let task_a = task::spawn(async move {
15         time::sleep(Duration::from_secs(3)).await;
16         let (resp_tx, resp_rx) = oneshot::channel();
17         if let Err(_) = tx1.send((50, resp_tx)).await {
18             println!("receiver dropped");
19             return;
20         }
21         if let Ok(ret) = resp_rx.await {
22             if ret {
23                 println!("task_a finished with success.");
24             } else {
25                 println!("task_a finished with failure.");
26             }
27         } else {
28             println!("oneshot sender dropped");
29             return;
30         }
31     });
32     let task_b = task::spawn(async move {
33         let (resp_tx, resp_rx) = oneshot::channel();
34         if let Err(_) = tx2.send((100, resp_tx)).await {
35             println!("receiver dropped");
36             return;
37         }
38         if let Ok(ret) = resp_rx.await {
39             if ret {
40                 println!("task_b finished with success.");
41             } else {
42                 println!("task_b finished with failure.");
43             }
44         } else {
45             println!("oneshot sender dropped");
46             return;
47         }
48     });
49
50     let task_c = task::spawn(async move {
51         while let Some((i, resp_tx)) = rx.recv().await {
```

```

52         println!("got = {}", i);
53         db[4] = i;
54         println!("{}", db);
55         resp_tx.send(true).unwrap();
56     }
57 });
58
59 _ = task_a.await.unwrap();
60 _ = task_b.await.unwrap();
61 _ = task_c.await.unwrap();
62 }
63 // 输出
64 got = 100
65 [1, 2, 3, 4, 100, 6, 7, 8, 9, 10]
66 task_b finished with success.
67 got = 50
68 [1, 2, 3, 4, 50, 6, 7, 8, 9, 10]
69 task_a finished with success.
70 ^C

```

解释一下这个例子，这个例子中的第 9 行，把消息类型定义成了 `(u32, oneshot::Sender<bool>)`。对，你没看错，是一个元组，元组的第二个元素为 oneshot channel 的发送端类型。

然后第 16 行，在 `task_a` 中创建了一个 Oneshot channel，两个端为 `resp_tx` 和 `resp_rx`。然后在第 17 行，把 `resp_tx` 实例直接放在消息中，随着 MPSC Channel 一起发送给 `task_c` 了。然后在 `task_a` 里用 `resp_rx` 等待 oneshot 通道的值传过来。这点很关键。`task_b` 也是类似的处理。

在 `task_c` 里，第 51 行收到的消息是 `Some((i, resp_tx))`，`task_c` 拿到了 `task_a` 和 `task_b` 里创建的 Oneshot channel 的发送端 `resp_tx`，就可以用它在第 55 行把计算的结果发送回去：`resp_tx.send(true).unwrap();`。

这个例子非常精彩，也是一种比较固定的模式。因为通道两个端本身就是类型的实例，当然可以被其他通道传输。这里我们 MPSC + Oneshot 两种通道成功实现了 Request/Response 模式。

tokio 中的其他 channel 类型

接下来我们再介绍一下 tokio 中的其他 channel 类型。tokio 中还有两个内置通道类型，用得不是那么多，但功能非常强大，你可以在遇到合适的场景时再去具体研究。

broadcast channel

广播模式，实现了 MPMC 模型，也就是多生产者多消费者模式，可以用来实现发布 - 订阅模式。每个消费者都会收到每个生产者发出的同样的消息副本。你可以查看 [🔗链接](#) 了解学习。

broadcast 通道实际已覆盖 SPMC 模型，所以不用再单独定义 SPMC 了。

watch channel

watch 通道实际是一个特定化版本的 broadcast 通道，它有 2 个特性。

1. 只有一个生产者，多个消费者。
2. 只关心最后一个值。

它适用于一些特定的场景，比如配置更新需要通知工作任务重新加载，平滑关闭任务等等。你可以通过我给出的 [🔗链接](#) 进一步学习。

补充知识：任务管理的 2 种常见模式

等待所有任务一起返回

前面示例中 task_c 很关键。为什么呢？因为它不但起到了搜集数据执行操作的作用，它还把整个程序阻塞住了，保证了程序的持续运行。那如果一个程序里面没有负责这个任务的角色，应该怎么去搜集其他任务返回的结果呢？我们在 [🔗第 13 讲](#) 中已经提到了一种方式。

 复制代码

```
1 use tokio::task;
2
3 async fn my_background_op(id: i32) -> String {
4     let s = format!("Starting background task {}.", id);
5     println!("{}", s);
6     s
7 }
```

```

8  #[tokio::main]
9  async fn main() {
10     let ops = vec![1, 2, 3];
11     let mut tasks = Vec::with_capacity(ops.len());
12     for op in ops {
13         // 任务创建后，立即开始运行，我们用一个Vec来持有各个任务的handler
14         tasks.push(tokio::spawn(my_background_op(op)));
15     }
16     let mut outputs = Vec::with_capacity(tasks.len());
17     for task in tasks {
18         // 在这里依次等待任务完成
19         outputs.push(task.await.unwrap());
20     }
21     println!("{:?}", outputs);
22 }
23

```

上面的代码有两个关键点。

1. 在第 15 行用一个 Vec 来存放所有任务的 handler。
2. 在第 20 行依次对 task 进行 await，获取任务的返回值。

这代表了一种模式。这个模式有个特点，就是要等待前面任务结束，才能拿到后面任务的返回结果。如果前面某个任务执行的时间比较长，即使后面的任务实际已经执行完了，在最后搜集结果的时候，还是需要等前面那个任务结束了后，我们才能搜集到后面任务的结果。比如：

 复制代码

```

1  use std::time::Duration;
2  use tokio::task;
3  use tokio::time;
4
5  #[tokio::main]
6  async fn main() {
7      let task_a = task::spawn(async move {
8          println!("in task_a");
9          time::sleep(Duration::from_secs(3)).await; // 等待3s
10         1
11     });
12     let task_b = task::spawn(async move {
13         println!("in task_b");
14         2
15     });

```

```

16     let task_c = task::spawn(async move {
17         println!("in task_c");
18         3
19     });
20
21     let mut tasks = Vec::with_capacity(3);
22     tasks.push(task_a);
23     tasks.push(task_b);
24     tasks.push(task_c);
25
26     let mut outputs = Vec::with_capacity(tasks.len());
27     for task in tasks {
28         println!("iterate task result..");
29         // 在这里依次等待任务完成
30         outputs.push(task.await.unwrap());
31     }
32     println!("{:?}", outputs);
33 }
34 // 输出
35 iterate task result..
36 in task_a
37 in task_b
38 in task_c    // 在这之后会等待 3 秒，然后继续打印
39 iterate task result..
40 iterate task result..
41 [1, 2, 3]

```

上面的示例创建了三个任务 `task_a`、`task_b`、`task_c`，在 `task_a` 里等待 3 秒返回，`task_b` 和 `task_c` 都是立即返回。执行的时候，当打印出 `"in task_c"` 后，会停止 3 秒左右，然后继续打印剩下的，印证了我们前面的分析。

`tokio` 提供了一个宏 `tokio::join!()`，用来简化上面代码的写法，表示等待所有任务完成后，一起返回一个结果。用法如下：

[复制代码](#)

```

1 use std::time::Duration;
2 use tokio::task;
3 use tokio::time;
4
5 #[tokio::main]
6 async fn main() {
7     let task_a = task::spawn(async move {
8         println!("in task_a");

```

```
9         time::sleep(Duration::from_secs(3)).await; // 等待3s
10     1
11 });
12 let task_b = task::spawn(async move {
13     println!("in task_b");
14     2
15 });
16 let task_c = task::spawn(async move {
17     println!("in task_c");
18     3
19 });
20
21 let (r1, r2, r3) = tokio::join!(task_a, task_b, task_c);
22
23 println!("{}", {}, {}, {}, r1.unwrap(), r2.unwrap(), r3.unwrap());
24 }
25 // 输出
26 in task_a
27 in task_b
28 in task_c
29 1, 2, 3
```

这两个示例基本等价，都是在所有任务中等待最长的那个任务执行完成后，统一返回。你可以想想为什么它们差不多。

等待其中一个任务先返回

在实际场景中，还有另外一大类需求，就是在一批任务中，哪个任务先执行完，就马上返回那个任务的结果。剩下的任务，要么是不关心它们的执行结果，要么是直接取消它们继续执行。

针对这种场景，tokio 提供了 `tokio::select!()` 宏。用法如下：

[复制代码](#)

```
1 use std::time::Duration;
2 use tokio::task;
3 use tokio::time;
4
5 #[tokio::main]
6 async fn main() {
7     let task_a = task::spawn(async move {
8         println!("in task_a");
9         time::sleep(Duration::from_secs(3)).await; // 等待3s
```

```
10         1
11     });
12     let task_b = task::spawn(async move {
13         println!("in task_b");
14         2
15     });
16     let task_c = task::spawn(async move {
17         println!("in task_c");
18         3
19     });
20
21     let ret = tokio::select! {
22         r = task_a => r.unwrap(),
23         r = task_b => r.unwrap(),
24         r = task_c => r.unwrap(),
25     };
26
27     println!("{}", ret);
28 }
29 // 输出
30 // 第一次
31 in task_b
32 in task_a
33 2
34 in task_c
35 // 第二次
36 in task_a
37 in task_c
38 in task_b
39 2
40 // 第n次
41 in task_a
42 in task_c
43 in task_b
44 3
```

请注意示例里第 21 行到第 25 行的写法，这是 `tokio::select!` 宏定义的语法，不是 Rust 标准语法。变量 `r` 表示任务的返回值。当你多次执行上面代码后，你会发现，输出结果并不固定，你可以想一下为什么会这样。

小结

这节课我们讨论了在 Rust 中如何应用 channel 这种编程范式，在并发编程中避免使用锁。Rust 的 `tokio` 库提供了常用的通道模型基础设施。

1. MPSC 多生产者，单消费者 channel
2. Oneshot 一次性 channel
3. broadcast 广播模式
4. watch 观察者模式

每种通道都有各自的用途，适用于不同的场景需求。这一讲我们重点讲解了前两种通道，只要你掌握了它们，另外两种使用方式也是差不多的。这节课讨论的这些模式相当固定，只要照搬套用就可以了。

本讲代码链接： <https://github.com/miketang84/jikeshijian/tree/master/16-channel>

思考题

你可以说一说从任务中搜集返回结果有几种方式吗？欢迎你把你对课程的思考和疑问留在评论区，我会和你一起交流探讨，如果你觉得这节课的内容对你有帮助的话，也欢迎你分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (8)

最新 精选



刘丹

2023-11-24 来自广东

请问老师： `r = task_a => r.unwrap()` 这是闭包，还是匿名函数？

作者回复：这是tokio那个宏里面的特殊语法，不是标准的rust语法。就是一个语法而已。既不是闭包，也不是匿名函数



1



superggn

2023-12-25 来自北京

思考题

```
- Arc::new(Mutex::new(target_var));  
- res = join_handler.await.unwrap();  
- channel
```

作者回复：棒棒哒



superggn

2023-12-25 来自北京

捉虫：`等待所有任务一起返回` main 前面少个 fn

作者回复：哈哈，感谢指出，可能编辑的时候漏了。尽快处理



刘永臣

2023-11-28 来自北京

.await()类似于等待组吧？ channel的四种模式也是go channel常用的四种场景。

作者回复：await是单个任务。rust中waitgroup实现也很简单。四种模式对应起来了，可能tokio就是借鉴的go channel。



学水

2023-11-25 来自加拿大

如果通道都没有任何生产者消息，select语句中的消费者是堵塞在那里还是会之间进入下一个语句呢

作者回复：rx.await就是在task层面阻塞住。





老大

2023-11-24 来自河南

为啥我按照你写的，运行不起来呢？

作者回复：你可以帖一下代码。



PETFiSh

2023-11-24 来自上海

从任务收集返回结果的方式有：

- 1、任务直接返回值，然后通过handler取回，比如：a = task_a.await.unwrap();
- 2、通过锁的方式直接写在目标位置
- 3、通过channel的形式传递结果
- 4、似乎也可以unsafe来写全局变量。

作者回复：👍，很棒。mpsc 和 oneshot 都可以派上用场。



伯阳

2023-11-24 来自北京

这种无锁并发，快是快了点，但是如果给通道打满了，怎么处理呢

作者回复：打满了就阻塞了。打不进去了，就等吧，消耗一个就能打进去一个。

